

Laporan Teknis TK 1 Analisis Numerik

Soal 1: Analisis Perpindahan Penumpang Antarhalte

Kelompok: _____

Anggota dan NPM: _____

Semester Gasal 2026/2027

2 Data dan Validasi

2.1 Deskripsi data

Sebuah koridor memiliki N halte bernomor $1, \dots, N$ sesuai urutan lokasinya. Misalkan X_k adalah halte tempat seorang penumpang berada pada interval waktu ke- k . Matriks transisi $T \in \mathbb{R}^{N \times N}$ berisi $T_{ij} = \Pr(X_{k+1} = j \mid X_k = i)$, yaitu peluang penumpang berpindah dari halte i ke halte j dalam satu interval. Data kode B terdiri atas enam berkas `T_N.csv` untuk $N \in \{16, 32, 64, 128, 256, 512\}$, masing-masing berisi N baris dan N kolom bilangan riil tanpa header. Pada keenam berkas, elemen tak-nol hanya terletak pada kolom $j \in \{i-1, i, i+1, i+2\}$. Dalam satu interval, penumpang hanya dapat mundur satu halte, tetap, atau maju satu hingga dua halte, sehingga setiap baris memuat dua sampai empat elemen tak-nol dan T berstruktur pita (*banded*).

2.2 Prosedur validasi

Setiap matriks dibaca dengan fungsi `baca_T` lalu diperiksa oleh `validasi_T` (Lampiran A). Pemeriksaan pertama menyangkut bentuk data: T harus berukuran $N \times N$ sesuai nama berkas, seluruh elemennya hingga (bukan NaN atau Inf), dan $T_{ij} \geq 0$ karena setiap elemen adalah peluang. Jumlah setiap baris harus satu. Data disimpan sebagai bilangan titik-mengambang, jadi syarat ini diperiksa dengan toleransi, yaitu $\max_i \left| \sum_{j=1}^N T_{ij} - 1 \right| \leq \tau$ dengan $\tau = 10^{-12}$. Nilai τ jauh di atas epsilon mesin presisi ganda, $\varepsilon = 2^{-53} \approx 1.11 \times 10^{-16}$ [3], tetapi cukup kecil untuk mendeteksi kesalahan data.

Syarat berikutnya menjamin distribusi *steady state* tunggal. Rantai Markov berhingga yang *irreducible*, yaitu setiap halte dapat dicapai dari setiap halte lain, memiliki tepat satu distribusi stasioner dan seluruh komponennya positif [1]. Untuk memeriksanya, T dipandang sebagai graf berarah G dengan simpul berupa halte dan sisi $i \rightarrow j$ jika $T_{ij} > 0$. Rantai *irreducible* jika dan hanya jika G terhubung kuat. Hal ini diperiksa dengan dua *breadth-first search* (BFS) [2] dari halte 1. BFS pertama dijalankan pada G untuk memastikan semua halte dapat dicapai dari halte 1, dan BFS kedua pada G dengan arah sisi dibalik untuk memastikan halte 1 dapat dicapai dari semua halte. Sebagai pemeriksaan tambahan, rantai juga harus aperiodik agar distribusi penumpang konvergen ke distribusi stasioner dari kondisi awal apa pun [1]. Untuk rantai *irreducible*, cukup ada satu $T_{ii} > 0$. Karena T disimpan dense, setiap BFS memindai satu baris atau kolom

penuhi untuk setiap simpul, sehingga biaya validasi $O(N^2)$. Biaya ini termasuk tahap persiapan data, bukan solver.

2.3 Hasil validasi

Tabel 2.1: Hasil validasi matriks transisi. Untuk semua N , dimensi sesuai, seluruh elemen hingga dan nonnegatif, rantai irreducible dan aperiodik, dan setiap baris memuat 2–4 elemen tak-nol.

N	$\max_i \sum_j T_{ij} - 1 $	$\min_i T_{ii}$	$\min\{T_{ij} > 0 : i \neq j\}$	Valid
16	1.11×10^{-16}	0.835	0.0144	Ya
32	2.22×10^{-16}	0.835	0.0147	Ya
64	2.22×10^{-16}	0.835	0.0148	Ya
128	2.22×10^{-16}	0.835	0.0149	Ya
256	2.22×10^{-16}	0.835	1.50×10^{-8}	Ya
512	2.22×10^{-16}	0.835	0.0150	Ya

Tabel 2.1 menunjukkan bahwa keenam matriks lolos seluruh pemeriksaan. Galat jumlah baris maksimum adalah ε untuk $N = 16$ dan $2\varepsilon = 2.22 \times 10^{-16}$ untuk ukuran lainnya, jadi selisih tersebut berasal dari pembulatan dan bukan dari kesalahan data. Seluruh elemen diagonal bernilai paling sedikit 0.835, sehingga setiap rantai aperiodik. Setiap T karena itu memiliki distribusi stasioner tunggal yang positif di semua halte. Sifat irreducible ini juga menjadi dasar bahwa matriks koefisien pada Bagian 3 nonsingular. Fungsi validasi juga diuji pada empat matriks 3×3 , yaitu satu matriks valid dan tiga matriks yang sengaja dibuat salah (ada elemen negatif, satu baris berjumlah 0.9, dan satu halte menyerap sehingga rantai tidak irreducible). Keempat keputusan sesuai harapan (`hasil/uji_validasi.csv`), jadi fungsi tersebut tidak sekadar meloloskan semua data.

Kolom keempat Tabel 2.1 memperlihatkan satu pengecualian. Pada $N = 256$, peluang pindah antara halte 128 dan 129 hanya berorde 10^{-8} , sedangkan peluang pindah positif terkecil pada matriks lain sekitar 0.014. Total peluang lintas dari halte 1–128 ke halte 129–256, yaitu $\sum_{i \leq 128 < j} T_{ij}$, hanya sekitar 9.0×10^{-8} , dan arah sebaliknya juga sekitar 9.0×10^{-8} . Matriks T_{256} tetap valid karena peluang itu positif dan rantai tetap irreducible. Namun, koridornya hampir terpecah menjadi dua blok. Dampaknya terhadap kondisi sistem linear dibahas pada Bagian 7.

3 Formulasi dan Penanganan Singularitas

Misalkan $\pi^{(k)} \in \mathbb{R}^N$ adalah vektor kolom dengan $\pi_i^{(k)} = \Pr(X_k = i)$. Hukum peluang total memberi $\pi_j^{(k+1)} = \sum_i \pi_i^{(k)} T_{ij}$, atau $\pi^{(k+1)} = T^\top \pi^{(k)}$. Distribusi steady state π adalah distribusi yang tidak berubah oleh satu langkah transisi, sehingga memenuhi

$$T^\top \pi = \pi, \quad \mathbf{1}^\top \pi = 1, \quad \pi_i \geq 0, \quad (3.1)$$

dengan $\mathbf{1} \in \mathbb{R}^N$ vektor yang seluruh elemennya satu. Dengan mendefinisikan $A = I - T^\top$, persamaan pertama menjadi $A\pi = 0$.

Matriks A singular. Setiap baris T berjumlah satu, yaitu $T\mathbf{1} = \mathbf{1}$, sehingga

$$\mathbf{1}^\top A = \mathbf{1}^\top - (T\mathbf{1})^\top = \mathbf{0}^\top.$$

Jumlah seluruh baris A adalah vektor nol. Baris-baris A bergantung linear dan $\det A = 0$, jadi eliminasi Gauss atau faktorisasi LU tidak dapat langsung dipakai pada A . Hubungan yang sama menunjukkan bahwa baris pertama A sama dengan $-\sum_{i \geq 2} A_{i,:}$. Persamaan pertama dari $A\pi = 0$ tidak membawa informasi baru [4], dan dapat diganti dengan syarat skala $z_1 = 1$:

$$B_{1,:} = e_1^\top, \quad B_{i,:} = A_{i,:} \ (i = 2, \dots, N), \quad b = e_1, \quad Bz = b, \quad (3.2)$$

dengan $e_1 = (1, 0, \dots, 0)^\top$.

Matriks B nonsingular. Ekspansi kofaktor sepanjang baris pertama memberi $\det B = \det A_{2:N, 2:N}$, yaitu determinan A setelah baris dan kolom halte 1 dibuang. Submatriks ini sama dengan $I - \tilde{T}^\top$, dengan $\tilde{T}$ matriks transisi tanpa halte 1. Karena rantai irreducible, penumpang di halte mana pun pada akhirnya mencapai halte 1. Peluang untuk terus berada di halte $2, \dots, N$ menuju nol, yaitu $\tilde{T}^k \rightarrow 0$. Akibatnya 1 bukan nilai eigen $\tilde{T}$, sehingga $I - \tilde{T}^\top$ dan B nonsingular [4].

Solusi z masih perlu dinormalkan. Distribusi tunggal π memenuhi $A_{i,:}\pi = 0$ untuk $i \geq 2$ dan $\pi_1 > 0$, jadi solusi tunggal $Bz = b$ adalah $z = \pi/\pi_1$. Baris pertama hanya memilih skala $z_1 = 1$, sehingga umumnya $\mathbf{1}^\top z \neq 1$. Normalisasi

$$\pi = \frac{z}{\mathbf{1}^\top z} \quad (3.3)$$

mengembalikan syarat $\mathbf{1}^\top \pi = 1$. Syarat $\pi_i \geq 0$ ikut terpenuhi karena $z = \pi/\pi_1 > 0$.

4 Identifikasi Struktur Banded

Untuk matriks $M \in \mathbb{R}^{N \times N}$, *lower bandwidth* p dan *upper bandwidth* q adalah

$$p = \max\{i - j : M_{ij} \neq 0\}, \quad q = \max\{j - i : M_{ij} \neq 0\}, \quad (4.1)$$

dengan nilai 0 jika himpunannya kosong. Matriks pita hanya memiliki elemen tak-nol pada p diagonal di bawah dan q diagonal di atas diagonal utama. Fungsi `deteksi_bandwidth` menghitung (4.1) secara otomatis dari posisi elemen tak-nol. Untuk keenam ukuran diperoleh $p = 2$ dan $q = 1$, baik untuk A maupun B . Jadi B *bukan* tridiagonal.

Struktur ini dapat diturunkan langsung. Untuk $i \geq 2$, $B_{ij} = \delta_{ij} - T_{ji}$ dengan δ_{ij} delta Kronecker, sehingga elemen tak-nol T pada posisi $j - i = d$ muncul di B pada posisi $j - i = -d$. Karena posisi tak-nol T adalah $d \in \{-1, 0, 1, 2\}$, baris $i \geq 2$ dari B hanya memuat

$$B_{i,i-2} = -T_{i-2,i}, \quad B_{i,i-1} = -T_{i-1,i}, \quad B_{ii} = 1 - T_{ii}, \quad B_{i,i+1} = -T_{i+1,i}. \quad (4.2)$$

Penggantian baris pertama dengan $e_1^\top$ tetap mempertahankan struktur pita. Vektor $e_1^\top$ hanya tak-nol pada diagonal, jadi penggantian itu dapat menghapus elemen tak-nol tetapi tidak pernah menambah elemen di luar pita.

Matriks dense membutuhkan N^2 bilangan, sedangkan pita B hanya $(p + q + 1)N$. Solver dengan *partial pivoting* membutuhkan sedikit ruang tambahan. Pertukaran baris k

dengan baris $r \leq k + p$ membawa elemen tak-nol sampai kolom $r + q \leq k + p + q$, sehingga upper bandwidth faktor U dapat naik dari q menjadi $p + q$ (*fill-in*), sementara lower bandwidth L tetap p [5]. Kami menyimpan B dalam array $AB \in \mathbb{R}^{(2p+q+1) \times N}$ dengan format penyimpanan pita LAPACK [6]: B_{ij} berada di $AB_{p+q+1+i-j, j}$, sehingga setiap diagonal B menjadi satu baris AB dan p baris teratas dicadangkan untuk fill-in. Untuk $p = 2$ dan $q = 1$, array ini berukuran $6 \times N$.

Tabel 4.1: Kebutuhan memori penyimpanan B dalam byte (`float64`, 8 byte per elemen): dense N^2 , pita $(p + q + 1)N$, dan pita dengan ruang fill-in $(2p + q + 1)N$. Rasio = dense dibagi pita dengan fill-in.

N	Dense	Pita	Pita + fill-in	Rasio
16	2 048	512	768	2.7
32	8 192	1 024	1 536	5.3
64	32 768	2 048	3 072	10.7
128	131 072	4 096	6 144	21.3
256	524 288	8 192	12 288	42.7
512	2 097 152	16 384	24 576	85.3

Tabel 4.1 menunjukkan bahwa rasio penyimpanan dense terhadap pita dengan ruang fill-in sama dengan $N/6$. Keunggulan penyimpanan pita tumbuh linear terhadap N . Pada $N = 512$, matriks dense membutuhkan 2 MiB, sedangkan pita hanya 24 KiB, atau sekitar 85 kali lebih kecil.

5 Metode Penyelesaian

Kedua solver diimplementasikan sendiri dalam `soal1/soal1.ipynb`. NumPy hanya dipakai untuk array, *indexing*, dan aritmetika dasar; tidak ada pustaka penyelesaian SPL atau faktorisasi di dalam solver. Fungsi `np.linalg.solve` hanya dipakai pada sel verifikasi yang diberi label terpisah. Matriks permutasi P disimpan sebagai vektor indeks.

5.1 LU dense dengan partial pivoting

Solver (a) menghitung $PB = LU$ dengan L segitiga bawah berdiagonal satu, U segitiga atas, dan P matriks permutasi [3]. Pada langkah k , baris dengan $|B_{ik}|$ terbesar ($i \geq k$) ditukar ke posisi k . Pertukaran dilakukan pada seluruh baris, termasuk multiplier yang sudah tersimpan, agar L konsisten dengan P akhir. Multiplier $\ell_{ik} = B_{ik}/B_{kk}$ disimpan di tempat elemen yang dieliminasi, sehingga L dan U berbagi satu array $N \times N$. Sistem kemudian diselesaikan dengan substitusi maju $Ly = Pb$ dan substitusi mundur $Uz = y$. Langkah-langkah ini dirangkum pada Kode 5.1 dan diimplementasikan pada fungsi `lu_dense` dan `solve_lu`.

Kode 5.1 LU dense dengan *partial pivoting* dan penyelesaian $Bz = b$

Masukan: $B \in \mathbb{R}^{N \times N}$ nonsingular, $b \in \mathbb{R}^N$

Keluaran: $z \in \mathbb{R}^N$ dengan $Bz = b$

```
1:  $M \leftarrow B$  (salinan);  $perm \leftarrow (1, 2, \dots, N)$ 
2: for  $k = 1$  to  $N - 1$  do
3:    $r \leftarrow \arg \max_{k \leq i \leq N} |M_{ik}|$ ; if  $M_{rk} = 0$  then error “singular”
4:   Tukar baris  $M_{k,:} \leftrightarrow M_{r,:}$  dan  $perm_k \leftrightarrow perm_r$  ▷ seluruh baris
5:   for  $i = k + 1$  to  $N$  do
6:      $M_{ik} \leftarrow M_{ik}/M_{kk}$  ▷ multiplier  $\ell_{ik}$ 
7:      $M_{i,k+1:N} \leftarrow M_{i,k+1:N} - M_{ik} M_{k,k+1:N}$ 
8:   end for
9: end for
10:  $y \leftarrow b_{perm}$  ▷  $y = Pb$ 
11: for  $i = 2$  to  $N$  do ▷ substitusi maju  $Ly = Pb$ 
12:    $y_i \leftarrow y_i - \sum_{j < i} M_{ij} y_j$ 
13: end for
14: for  $i = N$  downto  $1$  do ▷ substitusi mundur  $Uz = y$ 
15:    $z_i \leftarrow (y_i - \sum_{j > i} M_{ij} z_j)/M_{ii}$ 
16: end for
17: return  $z$ 
```

5.2 Solver banded bergaya Thomas

Solver (b) harus berjalan dalam $O(N)$ waktu dan memori. Algoritma Thomas klasik memenuhi syarat ini, tetapi hanya berlaku untuk matriks tridiagonal ($p = q = 1$), sedangkan B memiliki $p = 2$ dan $q = 1$. Solver (b) karena itu berupa perluasannya untuk matriks pita umum, yaitu eliminasi Gauss dengan partial pivoting yang hanya bekerja di dalam pita [5], langsung pada array AB dari Bagian 4. Algoritmanya ada pada Kode 5.2 (fungsi `bentuk_B_band`, `lu_band`, dan `solve_band`).

Ada tiga hal yang membuat solver ini berjalan dalam $O(N)$. Pertama, pita diambil langsung dari T : fungsi `bentuk_B_band` mengisi AB menurut (4.2) dan hanya mengakses $N(p + q + 1)$ elemen T , jadi matriks B dense, L , U , maupun P berukuran $N \times N$ tidak pernah dibentuk. Kedua, pivot hanya dicari di baris $k, \dots, \min(N, k + p)$ karena di bawahnya kolom k sudah nol, sedangkan pertukaran dan pembaruan hanya menyentuh kolom $k, \dots, \min(N, k + p + q)$. Ketiga, multiplier langkah sebelumnya tidak ikut ditukar seperti pada Kode 5.1, karena tersimpan di kolom lain AB . Indeks pertukaran setiap langkah dicatat di vektor *piv*, lalu diulang dengan urutan yang sama pada ruas kanan saat substitusi maju. Hasilnya setara dengan $PB = LU$.

Kode 5.2 Faktorisasi dan penyelesaian banded dengan *partial pivoting*. Notasi B_{ij} berarti elemen yang disimpan di $AB_{p+q+1+i-j, j}$.

Masukan: $AB \in \mathbb{R}^{(2p+q+1) \times N}$ berisi pita B , $p, q, b \in \mathbb{R}^N$

Keluaran: $z \in \mathbb{R}^N$ dengan $Bz = b$

```

1: for  $k = 1$  to  $N$  do
2:    $i_{\max} \leftarrow \min(N, k + p); \quad j_{\max} \leftarrow \min(N, k + p + q)$ 
3:    $r \leftarrow \arg \max_{k \leq i \leq i_{\max}} |B_{ik}|$ ; if  $B_{rk} = 0$  then error “singular”
4:    $piv_k \leftarrow r$ ; tukar  $B_{kj} \leftrightarrow B_{rj}$  untuk  $j = k, \dots, j_{\max}$ 
5:   for  $i = k + 1$  to  $i_{\max}$  do
6:      $B_{ik} \leftarrow B_{ik}/B_{kk}$  ▷ multiplier  $\ell_{ik}$ 
7:      $B_{ij} \leftarrow B_{ij} - B_{ik}B_{kj}$  untuk  $j = k + 1, \dots, j_{\max}$ 
8:   end for
9: end for
10:  $y \leftarrow b$ 
11: for  $k = 1$  to  $N$  do ▷ maju: ulangi pertukaran dan eliminasi
12:   tukar  $y_k \leftrightarrow y_{piv_k}$ ;  $y_i \leftarrow y_i - B_{ik}y_k$  untuk  $i = k + 1, \dots, \min(N, k + p)$ 
13: end for
14: for  $i = N$  downto  $1$  do ▷ mundur:  $U$  selebar  $p + q$ 
15:    $z_i \leftarrow (y_i - \sum_{j=i+1}^{\min(N, i+p+q)} B_{ij}z_j)/B_{ii}$ 
16: end for
17: return  $z$ 

```

5.3 Uji kebenaran

Sebelum dipakai pada data, kedua solver diuji pada matriks kecil yang hasilnya dapat dicek manual (Tabel 5.1). Pada uji 2, hitungan tangan dengan dua kali pertukaran baris memberi

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 0.25 & 1 & 0 \\ 0.5 & 2/3 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 8 & 7 & 9 \\ 0 & -0.75 & -1.25 \\ 0 & 0 & -2/3 \end{pmatrix}, \quad perm = (3, 1, 2),$$

dengan $perm$ urutan baris B setelah permutasi P . Kedua solver menghasilkan faktor yang sama persis.

Tabel 5.1: Uji kebenaran pada matriks kecil.

Kasus	Solver	Hasil
1. Matriks valid dan tiga matriks rusak 3×3 (Bagian 2)	validasi	keempat keputusan sesuai
2. $B = \begin{pmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{pmatrix}$, $b = (3, 7, 19)^T$	dense, banded ($p = q = 2$)	L, U, P sama dengan hitungan tangan; $\max PB - LU = 0$; $z = (1, -1, 2)^T$
3. Matriks 4×4 dengan $B_{11} = 0$	dense	baris 1 dan 2 ditukar; galat solusi 4.4×10^{-16}
4. Matriks pita 8×8 , $p = 2$, $q = 1$, diagonal diperkecil 100 kali	banded vs dense	7 pertukaran baris, 8 elemen fill-in, U melebar tepat sampai $p + q = 3$; P dan U identik dengan solver dense; galat solusi 5.3×10^{-15}
5. Matriks singular $\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$	dense, banded	keduanya menolak karena pivot nol
6. Rantai <i>birth-death</i> 3×3 dengan $\pi = (\frac{1}{4}, \frac{1}{2}, \frac{1}{4})^T$	dense, banded	π tepat sama

Pada keenam data, pivoting tidak pernah menukar baris (`hasil/akurasi_dense.csv` dan `hasil/akurasi_band.csv`). Penyebabnya, B dominan diagonal per kolom. Pada kolom $k \geq 2$ berlaku $|B_{kk}| = 1 - T_{kk} = \sum_{i \neq k} T_{ki} \geq \sum_{i \neq k, i \geq 2} |B_{ik}|$, dan pada kolom pertama $B_{11} = 1 \geq T_{12} + T_{13}$. Eliminasi Gauss mempertahankan sifat ini, sehingga partial pivoting selalu memilih elemen diagonal [5]. Jalur pertukaran baris karena itu hanya teruji lewat uji 2–4 pada Tabel 5.1.

Tanpa pertukaran baris, kedua solver melakukan operasi titik-mengambang yang sama pada angka yang sama; solver dense hanya menambah operasi terhadap elemen nol. Hasilnya, π dari solver banded identik dengan π dari solver dense, dengan selisih maksimum tepat 0 untuk semua N . Sebagai pembanding independen, selisih terhadap `np.linalg.solve` paling besar 2.6×10^{-16} untuk $N \neq 256$ dan 6.6×10^{-13} untuk $N = 256$ (`hasil/verifikasi*_numpy.csv`). Selisih yang lebih besar pada $N = 256$ sejalan dengan struktur yang hampir terpecah pada Subbagian 2.3 dan dianalisis pada Bagian 7.

6 Eksperimen Kinerja dan Analisis Kompleksitas

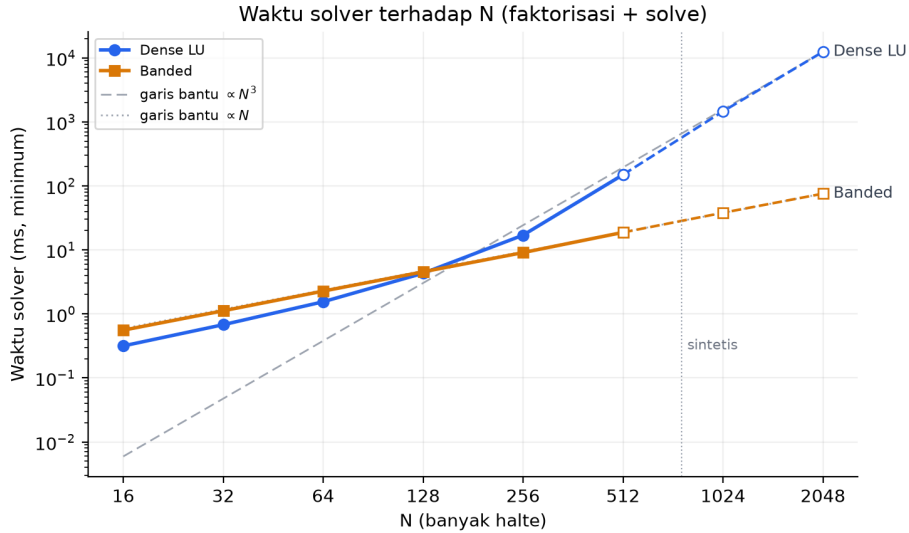
6.1 Rancangan eksperimen

Eksperimen dijalankan pada AMD Ryzen 7 7435HS dengan RAM 16 GB, Windows 11, Python 3.14.4, dan NumPy 2.5.3. Waktu solver adalah waktu faktorisasi, substitusi, dan normalisasi (3.3). Waktu membaca CSV dan persiapan diukur terpisah. Persiapan solver dense adalah pembentukan B dense, yang berbiaya $O(N^2)$. Persiapan solver banded adalah deteksi p dan q dari posisi tak-nol T , yang juga $O(N^2)$ karena data masukan dense, ditambah ekstraksi pita yang $O(N)$. Waktu diukur dengan `time.perf_counter`. Setelah satu kali pemanasan, setiap pengukuran diulang 15 kali dan diambil nilai minimumnya, karena gangguan dari sistem operasi hanya dapat menambah waktu. Memori diukur dengan dua cara: puncak alokasi selama solver berjalan menurut `tracemalloc`, dan total `.nbytes` array yang disimpan solver (koefisien, ruas kanan, faktor, vektor pivot, dan solusi). Seluruh hasil tersimpan di `hasil/benchmark.csv`.

6.2 Hasil

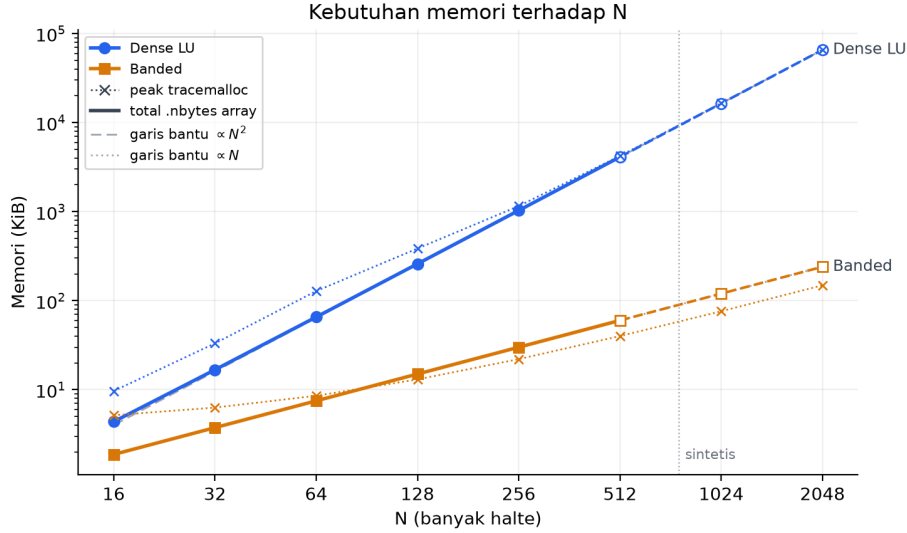
Tabel 6.1: Waktu (ms, minimum dari 15 ulangan) dan memori (KiB) untuk data soal.

N	Baca CSV	Dense (ms)		Banded (ms)		.nbytes		Peak	
		persiapan	solver	persiapan	solver	dense	banded	dense	banded
16	0.37	0.01	0.32	0.04	0.56	4.4	1.9	9.7	5.2
32	0.46	0.01	0.68	0.08	1.12	16.8	3.8	33.2	6.3
64	0.79	0.01	1.55	0.16	2.27	65.5	7.5	128.2	8.5
128	2.02	0.05	4.31	0.37	4.54	259.0	15.0	385.1	13.0
256	6.89	0.23	16.98	0.96	9.07	1030.0	30.0	1153.6	22.0
512	27.10	3.15	149.48	2.92	18.76	4108.0	60.0	4225.9	40.0



Gambar 6.1: Waktu solver terhadap N (skala log-log). Marker kosong dan garis putus-putus adalah rantai sintetis $N = 1024$ dan 2048 ; garis abu-abu adalah garis bantu $\propto N^3$ dan $\propto N$.

Tabel 6.1 dan Gambar 6.1 menunjukkan bahwa keunggulan solver banded baru muncul pada N sedang. Untuk $N \leq 64$, solver banded 1.5–1.8 kali lebih lambat daripada solver dense, dan pada $N = 128$ keduanya setara. Setiap langkah solver banded berupa perulangan Python kecil dengan biaya tetap, sedangkan setiap langkah solver dense adalah satu operasi vektor NumPy yang dieksekusi dalam kode terkompilasi. Pada N kecil, waktu lebih ditentukan oleh banyaknya langkah daripada banyaknya flop. Mulai $N = 256$ solver banded lebih cepat, yaitu 1.9 kali pada $N = 256$ dan 8.0 kali pada $N = 512$. Pada $N = 512$, waktu membaca CSV (27.1 ms) bahkan sudah melebihi waktu solver banded (18.8 ms). Untuk melihat tren pada N yang lebih besar, kedua solver juga dijalankan pada rantai sintetis $N = 1024$ dan 2048 dengan pola pita yang sama (bukan data soal, lima ulangan). Pada $N = 2048$, solver dense membutuhkan 12.4 s, sedangkan solver banded hanya 76 ms.



Gambar 6.2: Memori terhadap N . Garis tebal menunjukkan total `.nbytes` array yang disimpan solver; garis bertitik dengan marker silang menunjukkan puncak `tracemalloc`.

Perbedaan memori pada Gambar 6.2 lebih tegas daripada perbedaan waktu. Total `.nbytes` solver dense naik empat kali setiap N digandakan, sedangkan solver banded hanya naik dua kali. Pada $N = 512$, solver dense menyimpan 4108 KiB dan solver banded 60 KiB, sekitar 68 kali lebih kecil. Puncak `tracemalloc` mengikuti pola yang sama.

6.3 Analisis kompleksitas

Banyaknya operasi dihitung dalam flop: satu flop adalah satu pasangan perkalian dan pengurangan, atau satu pembagian. Pada solver dense, langkah k mengeliminasi $N - k$ baris dan tiap baris memperbarui $N - k$ elemen, sehingga faktorisasi memerlukan $\sum_{k=1}^{N-1} (N - k)^2 \approx N^3/3$ flop [3]. Substitusi maju dan mundur masing-masing sekitar $N^2/2$ flop. Array faktornya berukuran N^2 . Jadi solver dense membutuhkan $O(N^3)$ waktu dan $O(N^2)$ memori.

Pada solver banded, setiap kolom hanya mengeliminasi p baris, dan setiap baris hanya memperbarui $p + q$ elemen karena faktor U melebar akibat pivoting. Faktorisasi memerlukan sekitar

$$N p (p + q) \quad (6.1)$$

flop, yaitu $6N$ untuk $p = 2$ dan $q = 1$; taksiran $(p + q)^2 N/4$ yang lebih kecil berlaku untuk eliminasi pita tanpa pivoting. Substitusi maju dan mundur hanya menyentuh elemen di dalam pita, sehingga biayanya $O(N(p + q))$. Memori yang dibutuhkan adalah array AB berukuran $(2p + q + 1)N$. Untuk p dan q tetap, waktu dan memori sama-sama $O(N)$, sesuai yang dituntut dari solver (b). Sebagai gambaran, pada $N = 512$ faktorisasi dense memerlukan sekitar 4.5×10^7 flop, sedangkan faktorisasi banded hanya 3072 flop (`hasil/flop_teorii.csv`).

Tabel 6.2: Kemiringan garis log-log (pangkat empiris N) terhadap nilai teoretis. Kolom terakhir memakai $N = 256, 512$ dan dua titik sintetis.

Besaran	Teori	$N = 16-512$	$N = 128-512$	$N = 256-2048$
Waktu solver dense	3	1.71	2.56	3.18
Waktu solver banded	1	1.01	1.02	1.02
Flop faktorisasi dense	3	3.00	3.00	3.00
Flop faktorisasi banded	1	1.00	1.00	1.00
<code>.nbytes</code> dense	2	1.98	1.99	2.00
<code>.nbytes</code> banded	1	1.00	1.00	1.00
Peak dense	2	1.74	1.73	1.95
Peak banded	1	0.59	0.81	0.92

Tabel 6.2 membandingkan teori dengan eksperimen melalui kemiringan garis terbaik pada skala log-log, yang dihitung dengan rumus regresi linear sederhana. Kemiringan waktu solver banded sekitar 1 pada semua rentang, sesuai $O(N)$. Kemiringan waktu solver dense hanya 1.71 pada seluruh data karena pada N kecil biaya tetap setiap langkah masih dominan, lalu naik menjadi 2.56 untuk $N \geq 128$ dan 3.18 untuk $N \geq 256$, mendekati N^3 . Kemiringan flop faktorisasi dan `.nbytes` sama dengan teori: 3 dan 2 untuk dense, 1 untuk banded. Eksperimen ini mengonfirmasi analisis di atas: solver banded linear dalam waktu dan memori, sedangkan solver dense kuadratik dalam memori dan mendekati kubik dalam waktu.

Referensi

- [1] J. R. Norris, *Markov Chains*. Cambridge University Press, 1997, Bab 1.7–1.8.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, *Introduction to Algorithms*, edisi ke-3. MIT Press, 2009.
- [3] M. T. Heath, *Scientific Computing: An Introductory Survey*, edisi ke-2. McGraw-Hill, 2002, Bab 1–2.
- [4] W. J. Stewart, *Introduction to the Numerical Solution of Markov Chains*. Princeton University Press, 1994.
- [5] G. H. Golub dan C. F. Van Loan, *Matrix Computations*, edisi ke-4. Johns Hopkins University Press, 2013, Bab 3 dan Subbab 4.3.
- [6] E. Anderson dkk., *LAPACK Users' Guide*, edisi ke-3. SIAM, 1999.

A Kode Program

Program lengkap berada di `soal1/soal1.ipynb` dan dijalankan dengan *Restart & Run All*; seluruh tabel dan grafik disimpan ke `soal1/hasil/`. Berikut fungsi inti validasi, kedua solver, dan pengukuran waktu, disalin apa adanya dari notebook.

```

def bfs_jumlah_dikunjungi(T: np.ndarray, balik: bool = False) -> int:
    N = T.shape[0]
    dikunjungi = np.zeros(N, dtype=bool)
    dikunjungi[0] = True
    antrean = deque([0])
    while antrean:
        u = antrean.popleft()
        # graf asli: tetangga dari baris u; graf dibalik: dari kolom u
        if balik:
            tetangga = np.nonzero(T[:, u] > 0)[0]
        else:
            tetangga = np.nonzero(T[u, :] > 0)[0]
        for v in tetangga:
            if not dikunjungi[v]:
                dikunjungi[v] = True
                antrean.append(v)
    return int(dikunjungi.sum())

def validasi_T(T: np.ndarray, N: int, tol: float = TOL_BARIS) -> dict:
    hasil = {'N': N}
    hasil['dimensi_ok'] = T.shape == (N, N)
    hasil['hingga'] = bool(np.all(np.isfinite(T)))
    if not (hasil['dimensi_ok'] and hasil['hingga']):
        hasil['valid'] = False
        return hasil

    hasil['nonnegatif'] = bool(T.min() >= 0)
    hasil['galat_baris_maks'] = float(np.max(np.abs(T.sum(axis=1) - 1)))
    hasil['baris_ok'] = hasil['galat_baris_maks'] <= tol
    # terhubung kuat, semua halte tercapai dari halte 1 DAN halte 1 tercapai dari semua halte
    hasil['irreducible'] = (bfs_jumlah_dikunjungi(T) == N
                           and bfs_jumlah_dikunjungi(T, balik=True) == N)
    hasil['aperiodik'] = bool(np.any(np.diag(T) > 0))
    hasil['min_diag'] = float(np.diag(T).min())
    # peluang pindah terkecil yang masih positif; sangat kecil = rantai "hampir terputus"
    luar_diag = T - np.diag(np.diag(T))
    hasil['min_taknol_luar_diag'] = float(luar_diag[luar_diag > 0].min()) if np.any(luar_diag > 0) else 0.0

    nnz = np.count_nonzero(T, axis=1)
    hasil['nnz_per_baris'] = f'{nnz.min()}--{nnz.max()}'
    i, j = np.nonzero(T)
    hasil['offset_taknol'] = str(sorted(set((j - i).tolist()))) # nilai j - i dari elemen tak-nol

    hasil['valid'] = hasil['nonnegatif'] and hasil['baris_ok'] and hasil['irreducible']
    return hasil

def bentuk_B_dense(T: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    N = T.shape[0]
    B = np.eye(N) - T.T          # A = I - T^T
    B[0, :] = 0.0                # baris 1 diganti [1 0 ... 0]
    B[0, 0] = 1.0
    b = np.zeros(N)
    b[0] = 1.0
    return B, b

def deteksi_bandwidth(M: np.ndarray) -> tuple[int, int]:
    '''Menentukan bandwidth dari posisi elemen tak-nol.

    Input : M (N x N).
    Output: p = maks(i - j) (lower), q = maks(j - i) (upper).
    '''
    i, j = np.nonzero(M)
    p = max(0, int(np.max(i - j)))
    q = max(0, int(np.max(j - i)))
    return p, q

def lu_dense(B: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    LU = B.astype(float).copy()
    N = LU.shape[0]
    perm = np.arange(N)
    for k in range(N - 1):

```

```

    # pivot: elemen dengan nilai mutlak terbesar di kolom k, baris k..N-1
    r = k + int(np.argmax(np.abs(LU[k:, k])))
    if LU[r, k] == 0.0:
        raise ValueError(f'B singular: kolom {k} tidak punya pivot tak-nol')
    if r != k:
        LU[[k, r], :] = LU[[r, k], :] # tukar seluruh baris, termasuk multiplier lama
        perm[[k, r]] = perm[[r, k]]
    # multiplier l_ik disimpan di tempat elemen yang dieliminasi
    LU[k + 1:, k] /= LU[k, k]
    # baris i dikurangi l_ik * baris k (kolom k+1 ke kanan)
    LU[k + 1:, k + 1:] -= np.outer(LU[k + 1:, k], LU[k, k + 1:])
    if LU[N - 1, N - 1] == 0.0:
        raise ValueError('B singular: pivot terakhir nol')
    return LU, perm

def solve_lu(LU: np.ndarray, perm: np.ndarray, b: np.ndarray) -> np.ndarray:
    N = LU.shape[0]
    y = b[perm].astype(float) # Pb
    for i in range(1, N): # substitusi maju Ly = Pb (diagonal L = 1)
        y[i] -= np.dot(LU[i, :i], y[:i])
    z = np.zeros(N)
    for i in range(N - 1, -1, -1): # substitusi mundur Uz = y
        z[i] = (y[i] - np.dot(LU[i, i + 1:], z[i + 1:])) / LU[i, i]
    return z

def deteksi_bandwidth_T(T: np.ndarray) -> tuple[int, int]:
    i, j = np.nonzero(T)
    pakai = j >= 1
    p = max(0, int(np.max(j[pakai] - i[pakai])))
    q = max(0, int(np.max(i[pakai] - j[pakai])))
    return p, q

def bentuk_B_band(T: np.ndarray, p: int, q: int) -> tuple[np.ndarray, np.ndarray]:
    N = T.shape[0]
    d = p + q # baris AB tempat diagonal utama
    AB = np.zeros((2 * p + q + 1, N))
    AB[d, 0] = 1.0 # baris 1 B = [1 0 ... 0]
    for i in range(1, N):
        for j in range(max(0, i - p), min(N - 1, i + q) + 1):
            AB[d + i - j, j] = (1.0 if i == j else 0.0) - T[j, i]
    b = np.zeros(N)
    b[0] = 1.0
    return AB, b

def lu_band(AB: np.ndarray, p: int, q: int) -> tuple[np.ndarray, np.ndarray]:
    AB = AB.copy()
    d = p + q
    N = AB.shape[1]
    piv = np.zeros(N, dtype=int)
    for k in range(N):
        i_maks = min(N - 1, k + p)
        j_maks = min(N - 1, k + p + q)
        r = k + int(np.argmax(np.abs(AB[d:d + i_maks - k + 1, k])))
        if AB[d + r - k, k] == 0.0:
            raise ValueError(f'B singular: kolom {k} tidak punya pivot tak-nol')
        piv[k] = r
        js = np.arange(k, j_maks + 1)
        if r != k:
            sementara = AB[d + k - js, js]
            AB[d + k - js, js] = AB[d + r - js, js]
            AB[d + r - js, js] = sementara
        js = js[1:]
        for i in range(k + 1, i_maks + 1):
            AB[d + i - k, k] /= AB[d, k]
            AB[d + i - js, js] -= AB[d + i - k, k] * AB[d + k - js, js]
    return AB, piv

def solve_band(AB: np.ndarray, piv: np.ndarray, p: int, q: int, b: np.ndarray) -> np.ndarray:
    d = p + q
    N = AB.shape[1]
    y = b.astype(float).copy()
    for k in range(N):
        r = piv[k]

```

```

        if r != k:
            y[k], y[r] = y[r], y[k]
        for i in range(k + 1, min(N - 1, k + p) + 1):
            y[i] -= AB[d + i - k, k] * y[k]
    z = np.zeros(N)
    for i in range(N - 1, -1, -1):
        js = np.arange(i + 1, min(N - 1, i + p + q) + 1)
        z[i] = (y[i] - np.dot(AB[d + i - js, js], z[js])) / AB[d, i]
    return z

def ukur_waktu(fungsi, ulang: int = ULANG) -> float:
    fungsi()
    waktu = []
    for _ in range(ulang):
        t0 = time.perf_counter()
        fungsi()
        waktu.append(time.perf_counter() - t0)
    return float(np.min(waktu))

def ukur_peak(fungsi):
    tracemalloc.start()
    hasil = fungsi()
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak, hasil

```